

# Six parts from foundation to launch

Complete UI, server-authoritative multiplayer, cinematic 3D and release verification

Build Mythic Arena as a modular monolith using React and TypeScript, Node and Express, Socket.IO, Redis and MongoDB. This is a proposed implementation contract, not a claim that the project or its tests are already complete. Each part has a working deliverable and an explicit gate before the next part begins.

## The six delivery gates

- 1. Foundation and game contract.** Freeze the rules, shared types, tooling, screen structure and design tokens.
- 2. Identity, cards and decks.** Deliver secure accounts and a persistent, validated deck builder.
- 3. Game engine and playable board.** Make a deterministic match work with complete controls and private views.
- 4. Reliable online multiplayer.** Add matchmaking, atomic actions, deadlines, reconnect and durable results.
- 5. Complete UX and cinematic 3D.** Connect real screens and add portals, card reveals, summons and fallbacks.
- 6. Release hardening and deployment.** Verify security, two-client journeys, failure recovery and production operations.

## Scope boundaries

Release scope: one casual 1v1 queue, practice bot, original mythic card catalog, deck editing, match history and a simple wins leaderboard. Ranked seasons, trading, monetization, chat, spectator mode, full replay playback and separate microservices are deferred. Do not show nonfunctional Ranked buttons as active controls.

## How to execute each part with Codex

Ask Codex to inspect the repository and its instructions first, implement only the current part, preserve unrelated changes, run its gate tests, and report files changed, commands, failures and remaining gaps. Record actual evidence in docs/part-N-report.md. Do not mark a part complete merely because the interface looks finished.

# Freeze the gameplay contract

Proposed defaults to remove ambiguity before the engine and UI are built

## Rules to record in RULES.md

**1. Deck and opening.** A deck contains 30 cards, at most two copies of a definition. All seeded cards are available to both players. Use at least 18 original definitions. Start at 20 health with five cards each; the server shuffles and selects the first player.

**2. Turns.** On each personal turn, increase maximum energy by one up to ten, refill energy, draw one card and reset attack availability. The first player skips the first turn draw. A turn ends on End Turn or a 30-second server deadline.

**3. Units and spells.** Units have cost, attack and health; each side has five unit slots. New units cannot attack until their next turn. Each unit attacks once per turn. Spells resolve instantly and enter discard; use a finite effect registry, never executable text from a database.

**4. Targets and damage.** Units can attack an enemy unit or hero unless an enemy Guard unit exists. Unit combat deals simultaneous damage; remove dead units after resolution. Hero attacks do not cause retaliation. Shield absorbs the next damage instance. Healing caps at initial maximum health.

**5. Limits and finish.** Hand limit is ten; extra draws discard the drawn card. An empty-deck draw deals increasing fatigue damage starting at one. Hero health at zero ends the game; simultaneous defeat is a draw. Explicit surrender is immediate. At turn 100, declare a draw.

**6. Disconnect.** Allow 30 seconds to reconnect without pausing turns. If one player remains offline past the deadline, they forfeit; if both are offline at resolution, abort with no leaderboard credit. State is restored before controls are enabled.

## Adjustments to the earlier architecture

A short-lived Redis lock alone is insufficient if its lease expires. Part 4 requires a version-checked atomic commit, including deduplication and result-outbox creation. Timers use stored deadlines and a worker, not key expiration as a reliable scheduler. Match intros do not consume the first turn: both clients become ready or the server start deadline applies.

# Contracts, tooling and app shell

Dependency: gameplay defaults agreed and committed before feature coding

## Implementation tasks in order

- 1. Create the workspace.** apps/web, apps/api, apps/worker, packages/shared, packages/game-engine, infra and docs. Use TypeScript strict mode and a supported Node LTS; pin compatible dependencies in the lockfile. Select Redux Toolkit for client state and avoid competing stores.
- 2. Bootstrap local services.** Compose starts Redis and a MongoDB replica set for later transactions. Add validated environment configuration, .env.example without secrets, readiness and liveness endpoints, graceful shutdown and structured request IDs.
- 3. Define shared contracts.** Zod schemas cover cards, decks, commands, errors and player-safe snapshots. Commands carry gameId, actionId, expectedVersion, type and payload. Identity comes from authentication, never an incoming userId.
- 4. Create the UI shell.** Routes: home, login/register, lobby, decks, match, history, leaderboard and settings. Add loading, empty, error and not-found states. Establish navy surfaces, gold actions, cyan focus and violet accents; labels never rely on color alone.
- 5. Prepare verification.** Scripts: lint, typecheck, test, test:integration, test:e2e and build. Keep browser tests separate from unit collection. CI runs static checks and unit tests immediately; configure isolated test data and repeatable seeds.

## Deliverables

README setup instructions; RULES.md; docs/DECISIONS.md; shared event schemas; environment validation; Compose configuration; reusable Button, Dialog, Toast, StatusBanner and skeleton components; basic route tests.

## Gate tests

A clean checkout installs and builds using documented commands. Invalid server configuration fails clearly without printing secrets. Unknown routes render a useful fallback. Keyboard navigation, visible focus and reduced-motion settings work in the shell. One unit test and one browser smoke test run successfully in CI.

## Do not proceed until

The rule contract is explicit, the scripts actually run, and a fresh setup reaches the lobby shell. No authentication or network game behavior is represented as already working. Handoff: record package choices, setup evidence and any unresolved rule decision.

# Authentication, cards and decks

Dependency: Part 1 contracts and development environment pass their gate

## Implementation tasks in order

- 1. Persist identity.** Users have normalized unique email, display name and password hash. Add register, login, refresh, logout and me routes. Use short-lived access tokens in memory and rotating refresh tokens in secure httpOnly cookies; store only refresh-token hashes server-side.
- 2. Protect access.** Apply input validation, login throttles, safe error messages and ownership checks. Define permitted frontend origins and CSRF protection for cookie-authenticated mutations. Logout revokes refresh sessions and closes their sockets when multiplayer is added.
- 3. Seed the catalog.** Store versioned definitions with original names, faction, cost, stats, effect key and art reference. Provide stable instance IDs when cards enter a match. Validate every effect key against the engine registry.
- 4. Build deck APIs.** GET /cards and GET/POST/PATCH/DELETE /decks. Enforce 30 cards, copy limits and owner authorization on the server. Match creation freezes a validated deck and rules version so later edits cannot change an active game.
- 5. Build working screens.** Login and register forms; card filtering and search; card-detail panel; create, rename, duplicate, save and delete deck flows. Show count, validation reasons, save-in-progress and unsaved-change protection. Preserve drafts after a failed request.

## Deliverables

Auth module and session model; catalog seed and indexes; deck repository; REST contract documentation; functional lobby deck selector; no-secrets sample environment and integration fixtures.

## Gate tests

Test duplicate registration, invalid credentials, expired and reused refresh tokens, logout, unauthenticated routes and another user attempting deck edits. Test unknown card IDs, 29/31-card decks and more than two copies. Reload a saved deck and verify persistence. Verify failed saves preserve edits and deletion requires confirmation.

## Do not proceed until

Two separate accounts can log in and save valid decks; authorization is verified at the API rather than only hidden in the UI. Every seeded card has a defined behavior, or is excluded from playable decks until Part 3 implements it.

# Deterministic engine and battle UI

Dependency: valid decks and card definitions from Part 2

## Implementation tasks in order

- 1. Write the pure engine.** Implement `createMatch`, `validateAction`, `applyAction`, `advanceTurn`, `resolveDeaths`, `determineOutcome` and `projectForPlayer`. Pass time and random seed as inputs; do not call a database, socket or browser inside rules functions.
- 2. Implement the rule registry.** Support units, damage spells, healing, Guard and Shield with the frozen rules. Build seeded test scenarios for fatigue, overflow, full boards, death resolution and surrender. Use a simple legal-action practice bot through the same engine.
- 3. Separate private data.** Engine state contains both hands and deck orders. Player projections expose only their hand, public boards, deck counts and opponent hand count. Do not include seed or hidden IDs in network-ready snapshots or analytics.
- 4. Build the tactical board.** Place health and energy above the hand without overlap; show both unit rows, timer, turn owner and End Turn. Provide click-to-select and target controls plus keyboard equivalents. Dragging is optional; legal targets and rule explanations are always available.
- 5. Prepare server integration.** A replaceable transport adapter feeds snapshots to the UI. Add pending, rejected and resync states now. Do not optimistically subtract health; play resolution animations from accepted transitions. Build victory, defeat, draw and surrender dialogs.

## Deliverables

Engine package with scenario fixtures, private-view serializer, bot practice mode, complete 2D match screen, component tests and rules-to-test checklist. No 3D asset is required to finish a game.

## Gate tests

Same seed and command sequence yield identical state. Rejected commands leave state unchanged. Every legal effect resolves correctly; no energy becomes negative. Test simultaneous unit deaths, summoning delay, shield, fatigue and turn limit. Ensure opponent secrets are absent from serialized projections. Complete a practice match using only a keyboard.

## Do not proceed until

A complete match reaches a correct outcome, all implemented effects have automated tests, and the board stays readable with maximum hand and board sizes. Practice controls must remain clearly labeled as practice, not online multiplayer.

# Matchmaking and synchronized play

Dependency: deterministic engine and player-safe snapshots pass Part 3

## Implementation tasks in order

- 1. Authenticate sockets.** Verify sessions, enforce expiry and match membership, and validate every payload. Define queue:join/leave, match:ready, game:command and state:request with acknowledgements and structured errors. Limit one controlling session per player and expose takeover explicitly.
- 2. Reserve matches atomically.** One casual queue checks deck validity and rejects duplicate entries. An atomic Redis operation reserves two users and creates an initializing match ID. A watchdog repairs abandoned reservations. Require readiness with a 10-second start deadline; abort unready matches without penalty.
- 3. Commit actions safely.** Read state, calculate a candidate, then atomically compare version, check actionId and commit state plus accepted-action record. Retry conflicts after revalidation. A short lock may reduce contention but is not the correctness guarantee. The same actionId with a different payload is rejected.
- 4. Recover from missed events.** Broadcast private views after commit; clients discard older versions and request a snapshot after gaps or reconnect. If broadcast fails after commit, retrying the same action returns its recorded result. An acknowledgement timeout is unknown, not an automatic failed move.
- 5. Run durable deadlines.** A worker reads due turn and disconnect deadlines from a sorted set, rechecks version and deadline, and submits an idempotent system action. Rebuild scheduled deadlines from active matches on recovery. At equal deadlines, apply disconnect resolution before turn expiry.
- 6. Finalize reliably.** Atomically write a terminal result outbox beside the game state. A retrying worker transactionally writes MongoDB match results and wins with a unique matchId receipt. Keep terminal state until persistence is confirmed; expose result-pending if MongoDB is down.

## Gate tests and handoff

Two authenticated browsers complete a match. Concurrent duplicate moves apply once; conflicting distinct moves cannot spend the same energy. Test join/cancel races, unauthorized room access, lost acknowledgements, stale timers, refresh, 29-second reconnect and forfeit. Kill the API before broadcast and retry; interrupt the result worker and verify no duplicate win. Record Redis-outage policy: reject new actions, never fall back to independent local state; unrecoverable active-state loss aborts without credit.

# Cinematic polish with usable controls

Dependency: real server snapshots and online lifecycle from Part 4

## Implementation tasks in order

- 1. Finish every real journey.** Connect lobby, queue cancellation, deck editing, battle, results, history, wins leaderboard and settings. Show connection status and pending results honestly. Add first-match guidance for energy, targets and End Turn; dismiss it without blocking the board.
- 2. Build the 3D lobby.** React Three Fiber and Drei render an original low-poly floating arena, rotating rune rings, stars, subtle fog and occasional lightning. Keep navigation and text in accessible HTML. Use gentle pointer parallax, not uncontrolled camera movement.
- 3. Create signature moments.** Portal and player crests on match found; cursor card tilt and reveal flip; legendary summon with sparks; short ultimate impact; victory crest. Use GSAP for cinematic sequences and Motion for ordinary UI transitions. The intro is skippable and never extends server deadlines.
- 4. Synchronize animation.** Consume unique accepted event IDs; do not replay full effects on snapshot restore. Damage values update from authoritative state. Merge or skip overdue visual effects after lag so players regain control promptly. Never cover timer, energy or legal targets with particles.
- 5. Add graphics controls.** High, Low and reduced-motion modes; mute starts enabled. Lazy-load the scene, cap render resolution, reuse geometry and pause hidden tabs. Handle WebGL failure with static artwork. Touch users get tap-to-inspect instead of hover-only information.
- 6. Check layouts.** Test desktop 1440x900, laptop 1280x720 and phone 390x844. Use stacked status bars and a horizontal hand on phones, with a full-size card detail sheet. No forced orientation, clipped action buttons or overlay-hidden game values.

## Gate tests and deliverables

A two-player match works with 3D disabled, slow asset loading and simulated WebGL failure. Reduced motion removes camera punches and ambience; keyboard focus and dialog dismissal remain correct. Proposed budgets: 60 fps target desktop, 30 fps Low, and at most 5 MB compressed initial optional scene. Measure on a documented device; if budgets fail, simplify assets. Deliver asset-license inventory, visual regression images, all screen-state checks and an event-to-animation map.

# Verification, deployment and handoff

Dependency: functional, accessible online game with optional 3D from Part 5

## Implementation tasks in order

- 1. Run the complete test matrix.** Unit rules, API ownership, Redis concurrency, browser two-player matches, reconnect, worker retries, responsive layouts and graphics fallbacks. Fail CI on lint, type, test or build failures; preserve traces and test evidence for failures.
- 2. Audit security and privacy.** Test malformed socket payloads, session expiry, cross-user access, rate limits, origin rules and private-hand leakage. Redact tokens and hidden game data from logs. Scan dependencies and verify secrets are absent from frontend bundles.
- 3. Load test a declared target.** Start with 100 simultaneous players across 50 matches for 15 minutes. Proposed goal: p95 command acknowledgement below 250 ms in one test region, no duplicate commits and no result loss. Record environment and observed numbers, not assumed scale.
- 4. Deploy staging first.** Static frontend plus long-running API and worker containers; managed MongoDB replica set and persistent Redis in the same region. Configure TLS, WebSocket upgrades, exact origins, secrets, backups, readiness, and migration/seed jobs separately from app startup.
- 5. Exercise operations.** Track active matches, queue wait, command latency, rejected actions, timer lag and outbox backlog. Drain new matches before deployment and reconnect existing players safely. Test container restart, database outage, restore and rollback with nonproduction data.
- 6. Publish evidence and demo.** README, architecture decisions, API/event contracts, deployment/runbook, QA report and known limitations. A five-minute demo shows matchmaking, a move, duplicate rejection, reconnect, 3D fallback and persisted history.

## Final release gate

No unresolved critical security issue, hidden-information leak, incorrect outcome or duplicate result. All required journeys pass in staging; rollback is documented and tested. Protect user data and get owner confirmation before production provisioning or paid resources. Mark remaining minor issues explicitly. If testing multiple realtime instances, add adapter and transport routing tests; do not call the MVP horizontally scalable without evidence.



# Keep each part verifiable

Six implementation parts; one repeatable quality process

## Required completion evidence for every part

- 1. Scope.** List completed requirements by identifier, modified files and deferred work. Compare against the preceding part before adding features.
- 2. Automation.** Record exact test commands and observed pass/fail totals. Fix regressions before advancing. A skipped or unavailable test is not a pass.
- 3. Manual check.** Save the tested journey, browser, viewport and outcome. For visual changes, inspect the actual rendered interface at small and large sizes.
- 4. Failure check.** Demonstrate at least one unhappy path introduced in that part, including recovery rather than just an error message.
- 5. Handoff.** Update setup notes and relevant contracts. Commit a clean milestone only after reviewing the diff; preserve unrelated user work.

## Copyable instruction for one implementation session

Implement Part [N] of the Mythic Arena Six Part Implementation Plan only. First inspect repository instructions, existing code, tests and the previous part report. Summarize gaps against this part, then implement its tasks in dependency order. Use the shared contracts, preserve existing behavior and add regression tests. Do not invent successful tests, build extra scope or expose private game data. Run the applicable checks, document actual evidence in docs/part-[N]-report.md, and report blockers and remaining work. Stop at this part's completion gate.

## Important interpretation

The plan makes quality measurable; no document can guarantee a bug-free implementation. The UI illustrations from the earlier blueprint are visual direction, not final pixel specifications. Keep the 3D ambition, but validate the real layouts and game state. The earlier MotionSites reference was not a verified template review; no specific template dependency is required for this implementation.

## Definition of finished

A new reviewer can set up or open the game, use two accounts, play a complete online match, recover from a short disconnect and see one durable result. The experience remains usable without 3D. The repository contains enough tests and documentation to explain why these behaviors are reliable.